Object Oriented Programming (CSEG1044)

Unit 06 – Lecture 04: Database Design + JDBC Layer

Tofik Ali

School of Computer Science, UPES Dehradun

February 27, 2026

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Learning Outcomes

- Convert your entity model into tables with keys and constraints.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Learning Outcomes

- Convert your entity model into tables with keys and constraints.
- Implement DAO CRUD using JDBC (`PreparedStatement` + `ResultSet` mapping).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Learning Outcomes

- Convert your entity model into tables with keys and constraints.
- Implement DAO CRUD using JDBC (`PreparedStatement` + `ResultSet` mapping).
- Complete Deliverable-3: schema script + CRUD for 1–2 core entities.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Why a database layer?

- UI-only data disappears when the app closes.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Why a database layer?

- UI-only data disappears when the app closes.
- DB gives persistence, querying, and multi-user consistency.

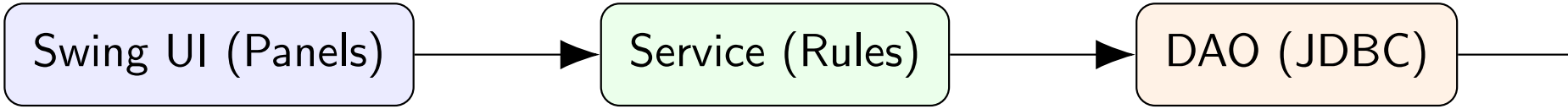
Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Why a database layer?

- UI-only data disappears when the app closes.
- DB gives persistence, querying, and multi-user consistency.
- Clean design: UI should *not* contain SQL.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

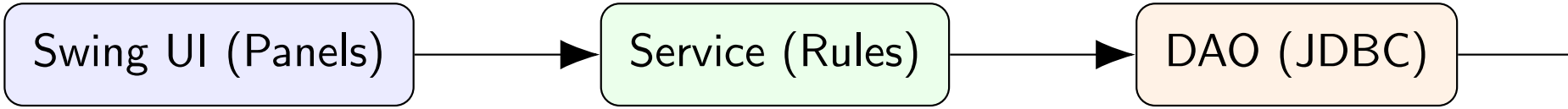
Capstone architecture (end-to-end)



- UI: read inputs, show messages.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

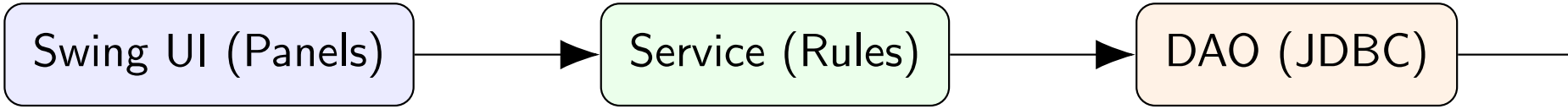
Capstone architecture (end-to-end)



- UI: read inputs, show messages.
- Service: validation + business rules.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Capstone architecture (end-to-end)



- UI: read inputs, show messages.
- Service: validation + business rules.
- DAO: SQL + mapping only.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Deliverable-3 (today)

- `schema.sql` committed to repo.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Deliverable-3 (today)

- `schema.sql` committed to repo.
- At least 1–2 entities with DAO CRUD (minimum: insert + list).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Deliverable-3 (today)

- `schema.sql` committed to repo.
- At least 1–2 entities with DAO CRUD (minimum: insert + list).
- A small runner (`main`) that proves DAO works.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Deliverable-3 (today)

- `schema.sql` committed to repo.
- At least 1–2 entities with DAO CRUD (minimum: insert + list).
- A small runner (`main`) that proves DAO works.
- Evidence: issues + PR + commit history.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Unit 06: Capstone Project

- Lecture focus: Database Design + JDBC Layer

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Today: Roadmap

- Entity model → relational schema (PK/FK, constraints)

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Today: Roadmap

- Entity model → relational schema (PK/FK, constraints)
- JDBC basics: connection, prepared statement, result set mapping

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Today: Roadmap

- Entity model → relational schema (PK/FK, constraints)
- JDBC basics: connection, prepared statement, result set mapping
- DAO pattern + deliverable checklist

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Entity model → tables (rule of thumb)

1. One entity class → one table.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Entity model → tables (rule of thumb)

1. One entity class → one table.
2. Fields → columns with types (`INT`, `VARCHAR`, `TIMESTAMP`).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Entity model → tables (rule of thumb)

1. One entity class → one table.
2. Fields → columns with types (`INT`, `VARCHAR`, `TIMESTAMP`).
3. 1-to-many → FK column on the "many" side.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Entity model → tables (rule of thumb)

1. One entity class → one table.
2. Fields → columns with types (`INT`, `VARCHAR`, `TIMESTAMP`).
3. 1-to-many → FK column on the "many" side.
4. Many-to-many → join table (two FKs).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Keys + constraints (minimum)

- **PK**: unique row id (often auto-increment integer).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Keys + constraints (minimum)

- **PK**: unique row id (often auto-increment integer).
- **FK**: reference to another table's PK (maintains relationship).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Keys + constraints (minimum)

- **PK**: unique row id (often auto-increment integer).
- **FK**: reference to another table's PK (maintains relationship).
- **NOT NULL**: required fields.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Keys + constraints (minimum)

- **PK**: unique row id (often auto-increment integer).
- **FK**: reference to another table's PK (maintains relationship).
- **NOT NULL**: required fields.
- **UNIQUE**: prevent duplicates (e.g., resource name).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Example schema (Resource Booking) (1/2)

```
CREATE TABLE resources (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  type VARCHAR(50) NOT NULL,
  capacity INT NOT NULL
);
```

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Example schema (Resource Booking) (2/2)

```
-- continued...
CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  resource_id INT NOT NULL,
  user_name VARCHAR(100) NOT NULL,
  start_time TIMESTAMP NOT NULL,
  end_time TIMESTAMP NOT NULL,
  status VARCHAR(20) NOT NULL,
  CONSTRAINT fk_booking_resource
    FOREIGN KEY (resource_id) REFERENCES resources(id)
);
```

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

JDBC building blocks (3 objects)

- `Connection`: session with DB.

Rule: close everything using `try-with-resources`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

JDBC building blocks (3 objects)

- `Connection`: session with DB.
- `PreparedStatement`: parameterized SQL (safe + reusable).

Rule: close everything using `try-with-resources`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

JDBC building blocks (3 objects)

- `Connection`: session with DB.
- `PreparedStatement`: parameterized SQL (safe + reusable).
- `ResultSet`: rows returned by a query.

Rule: close everything using `try-with-resources`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

PreparedStatement pattern (insert)

```
String sql = "INSERT INTO resources(name,type,capacity)
    VALUES(?,?,?)";
try (Connection con = DbUtil.getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, r.getName());
    ps.setString(2, r.getType());
    ps.setInt(3, r.getCapacity());
    ps.executeUpdate();
}
```

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Mapping ResultSet → objects (select)

```
String sql = "SELECT id,name,type,capacity FROM resources"
    ;
List<Resource> out = new ArrayList<>();
try (Connection con = DbUtil.getConnection();
     PreparedStatement ps = con.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        out.add(new Resource(
            rs.getInt("id"),
            rs.getString("name"),
            rs.getString("type"),
            rs.getInt("capacity")
        ));
    }
}
```

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs

When do you need transactions?

- When one user action triggers **multiple** SQL statements.

Idea: `con.setAutoCommit(false);` then `commit()/rollback()`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

When do you need transactions?

- When one user action triggers **multiple** SQL statements.
- Example: create booking **and** update resource availability.

Idea: `con.setAutoCommit(false);` then `commit()/rollback()`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

When do you need transactions?

- When one user action triggers **multiple** SQL statements.
- Example: create booking **and** update resource availability.
- If any step fails: rollback to keep DB consistent.

Idea: `con.setAutoCommit(false);` then `commit()/rollback()`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Common mistakes (avoid)

- Building SQL using string concatenation (SQL injection risk).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Common mistakes (avoid)

- Building SQL using string concatenation (SQL injection risk).
- Not closing `ResultSet`/`Statement`/`Connection`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Common mistakes (avoid)

- Building SQL using string concatenation (SQL injection risk).
- Not closing `ResultSet`/`Statement`/`Connection`.
- Putting SQL inside Swing button code (mixing layers).

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Common mistakes (avoid)

- Building SQL using string concatenation (SQL injection risk).
- Not closing `ResultSet`/`Statement`/`Connection`.
- Putting SQL inside Swing button code (mixing layers).
- Committing DB passwords in Git.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Demo plan (10–15 mins)

1. Run `schema.sql` to create tables.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Demo plan (10–15 mins)

1. Run `schema.sql` to create tables.
2. Run a tiny `main` that calls: `insert` then `findAll`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Demo plan (10–15 mins)

1. Run `schema.sql` to create tables.
2. Run a tiny `main` that calls: `insert` then `findAll`.
3. Verify in DB using `SELECT * FROM resources;`.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Mini Exercises (5 mins)

1. Add a UNIQUE constraint: which column in `resources` should be unique?

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Mini Exercises (5 mins)

1. Add a UNIQUE constraint: which column in `resources` should be unique?
2. Write SQL to list bookings for a resource id.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Mini Exercises (5 mins)

1. Add a UNIQUE constraint: which column in `resources` should be unique?
2. Write SQL to list bookings for a resource id.
3. In a DAO method, where does mapping happen: service or DAO?

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Key Takeaways

- Start from entities and design tables with keys + constraints.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Key Takeaways

- Start from entities and design tables with keys + constraints.
- DAO layer uses JDBC with prepared statements + mapping.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Key Takeaways

- Start from entities and design tables with keys + constraints.
- DAO layer uses JDBC with prepared statements + mapping.
- Deliverable-3 makes UI integration straightforward.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Checkpoint Questions

1. Why prefer `PreparedStatement` over string concatenation?

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Checkpoint Questions

1. Why prefer `PreparedStatement` over string concatenation?
2. What is the role of a foreign key?

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Checkpoint Questions

1. Why prefer `PreparedStatement` over string concatenation?
2. What is the role of a foreign key?
3. What should be inside a DAO method vs a service method?

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Next Lecture Preview

Next: Swing UI Panels + integration (Deliverable-4)

- Build screens/panels and connect them to your service/DAO.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

Next Lecture Preview

Next: Swing UI Panels + integration (Deliverable-4)

- Build screens/panels and connect them to your service/DAO.
- Implement one end-to-end flow: UI → DB → UI refresh.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).